

Agent Systems 研究型进阶专题 A · 第 3 讲

从稳定 Failure Pattern 到可证伪假设：第一次机制验证实验

本讲定位：第 1 讲解决“会跑真实 Benchmark”，第 2 讲解决“失败是否稳定”。第 3 讲开始做真正的科研：把重出的失模式成假，再用一受控干去。

1. 这一步为什么是“科研分水岭”

看到失不等于理解失。比如一 Agent 连续 4 次在退款任务上失败，我们至少有四种解释：它没读政策、形成正、工具用行了、或者用模器了不一致信息。“失了”“无法分些原因。

科研需要把模糊解释变成可证伪假设（falsifiable hypothesis）：如果某原因的是主要，那我们只针对这个原因施加干预时，结果应该朝一个可预测方向变化。



关键：不是“换个方法看看分数”，而是“如果 H 为真，干预后应该出现什么可预测变化？”

通俗比喻：医生看到“发烧”只是现象。给退烧药后体温下降，只能说明体温可以被药物改变；要判断病因，还要设计更有针对性的检查。Agent 失败机制实验也是同样逻辑。

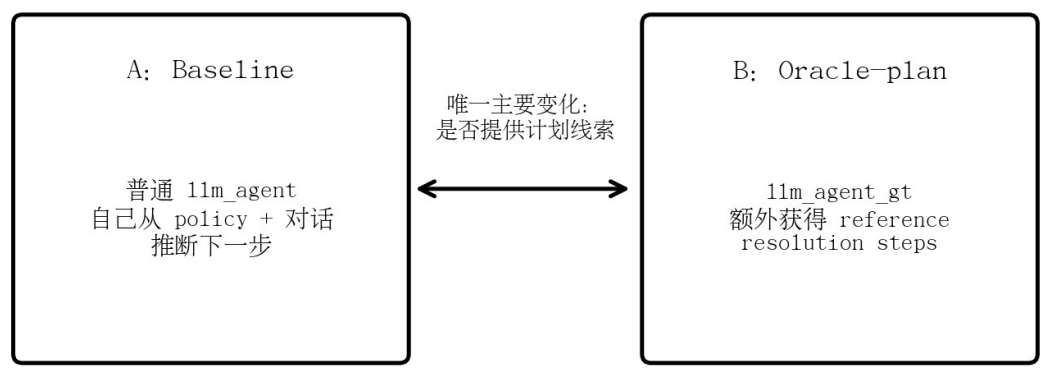
2. 本讲的研究问题

我们选择第 2 讲中“稳定失败”或“高失败率随机任务”，提出一个非常具体的第一假设：

H1：这些任务的主要瓶颈发生在“规划 / 下一步决策选择”阶段，而不是底层工具执行本身。

了 H1，我们立即自己改 Prompt，也不设计复杂新方法，而是先用 τ^3 -bench 官方已经提供的 LLMGTAgent（CLI 名称：llm_agent_gt）做一个“Oracle-plan intervention”。它会向 Agent 提供一条 reference resolution steps，使规划负担大幅降低。

3. 什么是 Oracle-plan 干预？



若 B 显著改善且 First Bad Span 后移/消失：支持“规划/决策是主要瓶颈”；但不是最终因果证明。

Baseline 条件使用普通 llm_agent：Agent 必须根据 policy、用户对话和工具反馈自行决定下一步。
Oracle-plan 条件使用 llm_agent_gt：它额外得到 benchmark 中的 reference resolution steps。

非常重要：Oracle-plan 不是“更公平的 Agent”，也不是可以拿去和排行榜直接比较的改进方法。它是一把诊断手术刀：我们故意把“规划难度”大幅削弱，观察失败是否随之消失。

4. 为什么这种设计比“换一个更强模型”更有信息量？

如果你直接把模型 A 换成更强的模型 B，几乎所有环节都同时改变：理解、规划、工具使用、语言表达、上下文能力甚至据。使分提高，也知道什。

而 Oracle-plan ablation 的思想是尽量只改变一个关键变量：是否额外提供计划线索。其余条件——domain、task IDs、agent model、user simulator、trial 数、总 seed、max steps——保持一致。

变量	Baseline	Intervention	是否应保持一致
Domain	retail	retail	是
Task IDs	同一组候选失败任务	同一组	是
Agent LLM	同一模型	同一模型	是
User LLM	同一模型	同一模型	是
num_trials	4	4	是
seed	300	300	是
Agent implementation	llm_agent	llm_agent_gt	唯一主要变化

5. 先学一个词：Ablation (消融)

Ablation 的核心不是“删掉模块”这一个动作，而是通过有控制的变化来判断某个成分是否对现象重要。在 Agent Systems 中，常见消融包括：去掉 memory、去掉 communication、给 oracle plan、限制工具、固定角色、改变 context 等。

科研表达：“B 比 A 高”只是结果；“在其他条件尽量固定时，提供 plan 信息后某类失败系统性减少”才开始接近机制据。

6. First Bad Span 如何进入机制分析？

第 2 讲已经把失败分成 stable fail / stochastic，并要求标 First Bad Span。第 3 讲不只看最终 reward，而要比对同一个 task 在两种条件下“第一个关键偏离点”发生在哪里。

Baseline First Bad Span	Oracle-plan First Bad Span	更合理的解释
计划选择错误	消失并最终成功	强烈支持规划瓶颈
计划选择错误	后移到工具参数错误	规划改善，但执行仍有第二瓶颈
工具结果理解错误	仍然相同	不支持“纯规划”解释
无明显 Agent 错误	仍失败	检查 user simulator、任务定义、评测或基础设施

7. pass^k 在 A/B 机制实验中的角色

只比较一次成功/失败会被随机性误导。本讲每个候选 task 默认在每个条件下跑 4 trials。你既要平均 success rate，也要看更严格的 pass^4：四次都成功的概率估计。

如果 Baseline 的某 task 是 1/4 成功，而 Oracle-plan 变成 4/4，解释价值远高于“Baseline 0/1，Intervention 1/1”。后者只有两个 episode，可能只是随机波动。

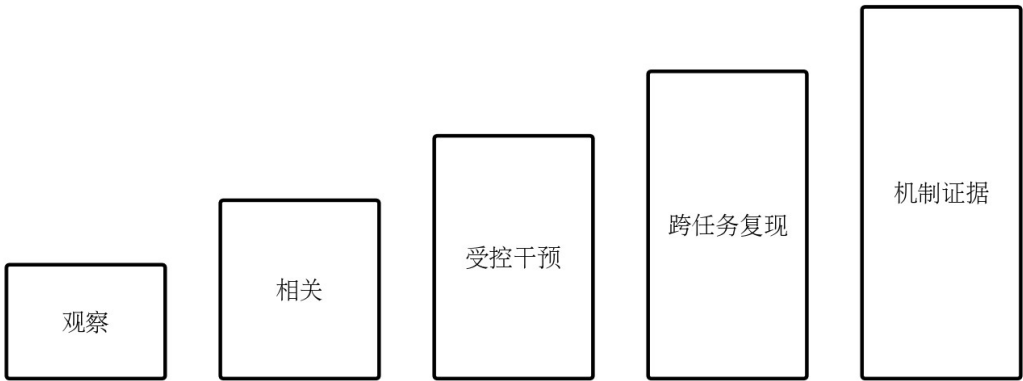
8. 结果怎么解释：不要抢着下结论

Baseline × Oracle-plan 结果解释矩阵

Baseline	Oracle-plan	优先解释
低	高	规划 / 决策瓶颈值得继续查
低	低	执行、通信、工具、任务/评测问题
高	高	该任务不适合做失败机制研究
高	低	异常结果：先排查参考轨迹/实现偏差

最有信息量的是“低 → 高”：Baseline 经常失败，而 Oracle-plan 在相同 task 上显著稳定地成功。这与“规划/决策瓶颈”假设一致。注意这里应写 supports / is consistent with，而不是 proves。原因是 oracle plan 同时可能减少搜索长度、降低上下文压力、减少用户交互轮数，所以它不是完美单因素实验。

机制结论的证据阶梯



本讲只走到 “受控干预”：结论应写成 supports / consistent with，而不是 proves。

9. 一个完整的教学例子

假设第 2 讲得到 Task 17：Baseline 4 次结果为 FAIL, FAIL, PASS, FAIL。阅读 Trace 后，3 个失败 episode 都在同一个位置提前执行了退款，而没有先确认资格。

于是提出 H1：主要瓶颈是“下一步决策/计划形成”，而不是 refund tool 本身。

干预后使用 llm_agent_gt，4 次结果变成 PASS, PASS, PASS, PASS，并且原来的 First Bad Span 消失。这个结果支持 H1。下一步才值得做更细的实验：不给完整 oracle plan，只加入“执行写操作前检查资格”的最小 verification checkpoint，看是否仍能复现改善。

为什么不直接写论文？：因为 Oracle-plan 太强，主要用于定位机制。正文通常需要一步缩小干预、跨更多 task/domain 复现，并排除替代解释。

10. 本讲真实实验流程

1. 从第 2 讲 task_stability.csv 选择 2-3 个 `stable_fail` 或 `success_rate ≤ 0.5` 的任务，并确认 task 的 `evaluation_criteria.actions` 非空（因为 `llm_agent_gt` 只适用于这类任务）。
2. 把候选 task IDs 写入环境变量 `TASK_IDS`，先运行 `baseline: llm_agent × 4 trials`。
3. 保持 task/model/user/seed 等不变，仅把 agent implementation 改成 `llm_agent_gt`。
4. 运行 `compare_conditions.py`，得到 `condition_summary.csv`、`task_condition.csv`、`mechanism_matrix.csv`。
5. 只对“Baseline 失败且 Oracle-plan 改善”的 episode 优先做 Trace 对照，标注 First Bad Span。
6. 填写 `hypothesis_card.md`：现象、假设、预测、干预、替代解释、结论强度。

11. 常见误区

- 误区 1：Oracle-plan 分数高，所以它是更好的最终 Agent。——错，它看到了额外 gold 信息，只能作伴。
- 误区 2：干预后成功就证明了规划是唯一原因。——错，干预可能同时缩短路径、降低上下文负担。
- 误区 3：只看总平均，不看 task-level。——错，机制通常首先出现在特定任务族。
- 误区 4：把 `infrastructure_error` 当模型失败。——继续分开统计。
- 误区 5：一次 A/B 就宣布机制。——至少要重复 trials，并最好跨多个 task 复现。

12. 本讲结束时你应该真正会什么

- 把“我觉得它是规划问题”改写成可证伪假设。
- 设计只改变一个主要因素的 `baseline/intervention` 对照。
- 理解 oracle intervention 公平性能比的。
- 把 `reward`、`pass^k`、`First Bad Span` 和 `Trace` 合成据。
- 用“支持/不支持/证据不足”而不是“证明了”来写机制结论。

13. 官方依据与版本说明

本讲以 `τ3-bench / tau2-bench v1.0.1` 为基准。官方源码中 `LLMAgent` 是普通半双工 Agent；`LLMGTAgent` 会接收 benchmark reference actions 转换出的 `resolution steps`，注册名为 `llm_agent_gt`。官方 evaluator 对 retail/airline/telecom 的主要正确性判断仍以 `reward basis`（通常 `DB + COMMUNICATE`）为准，因此本讲把 `oracle-plan` 作为诊断性 `ablation`，而不是排行榜性能结论。

参考：Sierra Research `tau2-bench` GitHub 仓库（v1.0.1），重点文件 `src/tau2/agent/llm_agent.py`、`src/tau2/registry.py`、`docs/evaluation.md`。